

PM PORTFOLIO · 2026

반짝이는 순간들 ✨

스쳐 지나가는 생각을 "결"별로 담아두는
감성 메모 앱 기획 · 설계 · 개발

발표자 조은정 · 기간 2026.04.20 ~ 2026.05.02 · 역할 기획 · UX · 개발 (1인)

CONTENTS

오늘 이야기의 흐름

- 01** **시작의 고백**
저는 다이어리를 1달 쓰고 11달 비웁니다
- 02** **문제 정의 & 가설**
왜 "아이디어"만 부담이 없을까?
- 03** **타겟 & 페르소나**
나 같은 사람이 또 있을까?
- 04** **제품 원칙**
우리가 절대 하지 않을 5가지
- 05** **솔루션 설계**
3개 화면, 7개의 결
- 06** **핵심 의사결정 6가지**
Trade-off로 증명하는 프로덕트 센스
- 07** **KPI & 측정**
복극성과 Anti-Metric
- 08** **회고 & 배움**
제품 + 기술, 다시 한다면
- 09** **AI 협업 투명성**
무엇을 맡기고, 무엇을 지켰나
- 10** **코드 × 의사결정**
3줄의 코드가 만드는 제품 밀도

“

저는 다이어리를
1달 쓰고 11달은 비워둡니다.

— 이 프로젝트는 여기서 시작됐습니다.

메모 앱, 다이어리, 노션 — 뭐든 한 달을 못 갑니다.
그런데 딱 하나, 부담 없이 적는 게 있었습니다.

왜 나는 계속 실패할까?

기록 앱에서 반복되는 나의 실패 패턴 3가지



다이어리

패턴

1월 폭발 → 2월 백지

진짜 이유

"매일 써야 한다"는 부담감



기본 메모장

패턴

쇼핑 리스트로만 가득

진짜 이유

"정리 없는 카오스" 공포



노션

패턴

템플릿 만들다 지쳐 종료

진짜 이유

"잘 써야 한다"는 완벽주의

공통점 → "잘 써야 한다"는 부담이 기록을 멈추게 한다.

딱 하나, 부담이 없던 단어

아이디어



아무때나 써도 된다



단어 하나 여도 된다



말이 안 돼도 된다



다시 안 봐도 된다

💡 발상: 이 "가벼움"을 모든 기록에 이식하면 어떨까?

"나 같은 사람이 또 있을까?"

주변 지인 10명 인터뷰 · 온라인 커뮤니티 리서치

"다이어리 사놓고 3일 썼어요"

"메모 앱 깔아놓고 안 켜요"

"노션은 너무 무거워서 포기"

"잘 쓰려다가 오히려 못 써요"

10명 중

8명

이 비슷한 경험을 가지고 있었다

— 이건 소수의 문제가 아니다

대부분의 메모 앱은 "기록을 잘 하는 사람"을 위해 설계되었다.
그런데 사용자의 다수는
"기록을 못 하는 사람"이다 → 시장의 공백

누구를 위한 앱인가

메인 타겟: "산만형 창작자" — 생각은 많은데 흘러보내는 20~30대



지수 (28)

마케터

"출근길에 떠오른 아이디어가 회사
가면 사라져요"

PAIN

아이폰 기본 메모 앱 — 찾기 너무 어려움

WANT

3초 안에 쓰고, 나중에 쉽게 찾고 싶음



현우 (32)

프리랜서 디자이너

"영감 노트 앱 5개 깔았지만 다 중간에
포기"

PAIN

노션 템플릿 만들다 지침, 에버노트 복잡함

WANT

예쁘게 쌓이고 감성적 회고 경험을 주는
도구



소희 (24)

대학원생

"일기 쓰고 싶은데 매일 쓸 자신이
없어요"

PAIN

종이 다이어리 여러 번 실패, 스트릭 앱은
스트레스

WANT

빈 날짜가 있어도 죄책감 없는 기록 공간

이 앱의 전제를 뒤집다

기 존 메 모 앱

"사용자는 정리를 잘 하고
싫어한다."



이 앱

"사용자는 정리를 못 하는
자신을 받아들이고 싶어한다."

검 증 할 핵 심 가 설

H1

"카테고리"라는 딱딱한 분류 대신 "결"이라는 감정 언어가 진입장벽을 낮춘다.

H2

"완벽하게 쓰려는 부담"이 메모 포기의 원인이다 → 질문 프롬프트 + 바로 저장 UX로 해소.

우리가 절대 하지 않을 5가지

기능 추가 시 1차 필터가 되는 제품 원칙

01

스트릭 카운터를 만들지 않는다

WHY · 연속 기록은 걱려가 아니라 부담이다

02

빈 날짜를 부정적으로 표시하지 않는다

WHY · 11달 비어도 괜찮다는 감각을 지킨다

03

분류를 강제하지 않는다

WHY · 기본값을 제공하고 바꾸고 싶을 때만 바꾼다

04

템플릿을 제공하지 않는다

WHY · 템플릿은 "이렇게 써야 한다"는 강요다

05

알림으로 재촉하지 않는다

WHY · 알림이 이탈의 1순위 원인이다

3개의 화면, 하나의 여정

기록 → 회고 → 정리 — 사용자의 자연스러운 흐름을 설계

0 1

기록 조각

기록 (Capture)

지금 떠오른 생각을 3초 안에
담는다

핵심 요소

- 랜덤 속삭임 프롬프트
- 바텀시트 카테고리
- 이미지 첨부

0 2

아이디어 달력

회고 (Revisit)

내가 언제 무슨 생각을 했는지
돌아본다

핵심 요소

- 날짜별 달력 뷰
- 조각 점 표시
- 빈 날짜 허용

0 3

마음 서랍

정리 (Archive)

비슷한 마음끼리 모아서 본다

핵심 요소

- 7개 결 그리드
- 카테고리 아카이브
- 최신순 정렬

7가지 "결" 시스템

생각을 감정·용도별로 나누는 7개 서랍



아이디어

반짝 떠오른 발상



영감

인용·발견·감탄



나에게 한마디

자기 확인·격려



중요

놓치면 안 되는



메모

일상 기록



할일

상태 관리 필요



기분

감정 일기

WHY 7?

밀러의 법칙 (7 ± 2)

인간이 단기 기억으로 다룰 수 있는 항목 수의 한계

3개 이하

세분화 부족 → 분류가 무의미해짐

10개 이상

선택 피로 → 분류를 포기함

7개

인지 최적점 · 감정 스펙트럼 충분

왜 "카테고리"가 아니라 "결"인가

용어 하나가 사용자의 태도를 바꾼다

PROBLEM

"카테고리를 선택하세요" — 업무적, 경직된 톤.
메모 앱을 일처럼 느끼게 만든다.

OPTIONS

- A "카테고리" 유지 — 관습적
- B "태그" — 중립, 일반적
- C "결" — 감성, 차별화**

DECISION

C. "결"

WHY (Rationale)

- 타겟 감성(20~30대)과 톤 일치 — 업무가 아닌 자기 대화
- 브랜드 톤("조각", "마음")과 일관된 어휘 생태계 구축
- "결" = 질감 + 흐름 + 감정 → 의미 확장성 우수

TRADE-OFF

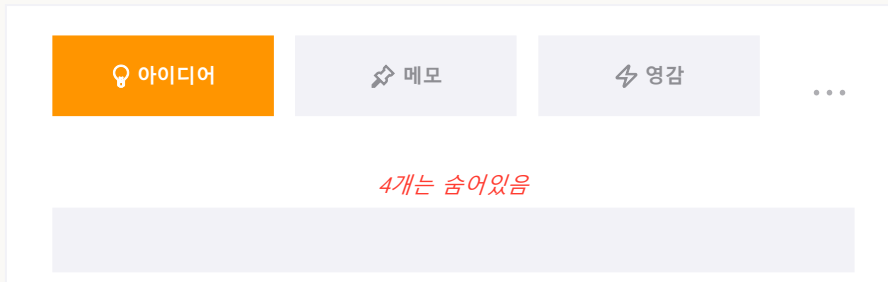
첫 사용자는 "결"의 의미를 바로 모를 수 있음 → 온보딩 1컷으로 보완 예정. 차별화 이익 > 학습 비용으로 판단.

[계획] 바텀시트 전환 예정

현재: catScroll 가로 칩 → 계획: 바텀시트 전환

BEFORE

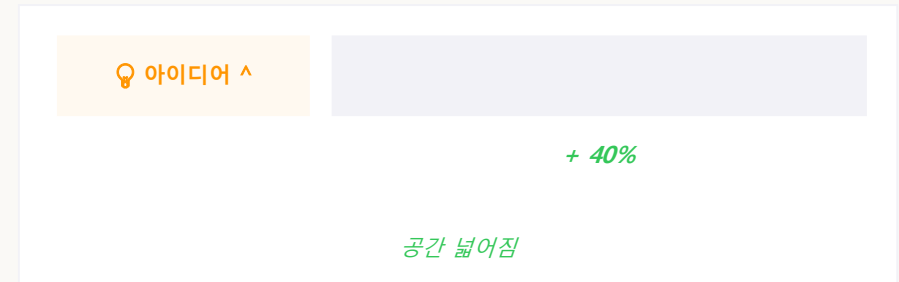
가로 스크롤 칩



- 7개 중 3~4개만 보임
- 끝 카테고리 선택률 낮음
- 타이핑하며 스크롤 동작 어색

AFTER

바텀시트 + 트리거 칩 하나



- ✓ 7개 전체 시각적 공평성
- ✓ 입력창 공간 +40% 확보
- ✓ EditModal과 UX 일관성

핵심: "보이는 것"이 "쓰이는 것"을 결정한다 — 발견 가능성(Discoverability) 중심의 설계

"할일"만 3단계 상태를 준 이유

조건부 UI — 기능을 언제 보여줄지가 프로덕트 센스다

❓ 질문

모든 조각(메모)에 "진행 상태"를 달면 어떨까?

X NO

모든 카테고리에 상태를 붙이면...

- "기분"에 "진행중"? → 어색함
- "영감"에 "완료"? → 의미 없음
- 모든 메모 = 작업 → 인지부담 ↑

✓ YES "할일"일 때만 조건부로 노출

```
107 const handleStatusChange = (newStatus: string) => {
108   setStatus(newStatus);
109
110   if (newStatus === '완료') {
111     const now = new Date();
112     setCompletedDate(
113       `${now.getFullYear()}.${now.getMonth() + 1}.${now.getDate()}`,
114     );
115   } else {
116     setCompletedDate(null);
117   }
118 };
119
120 const handleCategoryChange = (newCategory: string) => {
121   setCategory(newCategory);
122
123   if (newCategory !== '할일') {
124     setStatus('준비중');
125     setCompletedDate(null);
126   }
127 };
128
```

"기능은 많이 넣을 수 있다. 하지만 '언제 보여줄지'가 프로덕트 센스다."

→ 더하기보다 "숨기기"가 어렵다는 걸 배운 결정.

[계획] 메인+서브 결 구조

현재: subCategories 미구현 → v2.0 계획

PROBLEM · 실 사용 에서 발 견

한 조각이 여러 결에 걸치는 경우가 많다 — "엄마에게 전화"는 할일+기분, "좋은 말 들음"은 영감+중요

OPT I O N S

A 메인 1개 + 서브 여러개

(노션 스타일)

✓ 채택

B 제한적 복수 선택

(할일은 단독)

C 완전 복수 선택

(최대 자유)

W H Y — A 안 을 선택 한 4 가 지 이 유

- 메인 1개 유지 → 카드 UI 규칙(컬러, 라벨) 단순
- 할일 상태 UI 조건부 노출 로직 그대로 재사용
- subCategories 는 optional → 기존 데이터와 100% 호환
- 노션 Select/Multi-select 패턴 → 학습 비용 ↓

"단일 → 복수" 전환은 쉽다. 어려운 건 "메인이 무엇인지" 유지하는 규칙.

[계획] 오늘 날짜 필터링

"빠는 결정"이 제품의 밀도를 만든다 (v1.2)

PROBLEM · 과거가 현재를 방해한다

홈 화면에 전체 타임라인을 보여주면 — 앱을 열 때마다 과거 조각들이 시선을 잡아 "오늘 쓰는 것"에 집중이 흐려진다.

OPTIONS

A 전체 타임라인 노출

(일반적)

B 최근 7일만 노출

(절충안)

C 오늘 + 날짜 배너

(집중의 설계)

✓ 채택

WHY — 역할 분리가 만든 밀도

- 홈 = 오늘 전용 공간, 달력 = 과거 회고 공간 (역할 분리)
- 날짜 배너 + 주간 토글로 "다른 날로 이동"도 1탭 거리
- 메모가 쌓여도 홈 화면은 영원히 가볍게 유지됨
- 제품 원칙 2번 "빈 날짜를 부정적으로 표시하지 않는다"와 일관

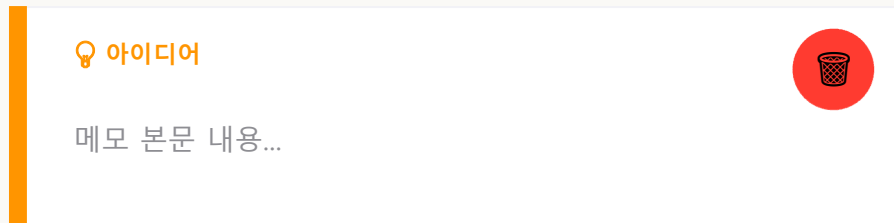
"더 보여주기"보다 "무엇을 감출지"가 PM의 실력이다.

[계획] 삭제 UX 설계

현재: 삭제 기능 미구현 → 계획: 롱프레스 삭제

BEFORE

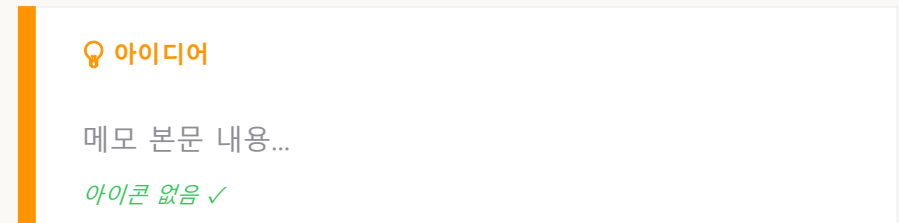
카드마다 쓰레기통 아이콘



- 실수로 누를 가능성 ↑
- 시각적 소음 증가
- 아이디어 앱에 위협적

AFTER

길게 누르기 + 편집 모달 내 버튼



- ✓ 길게 누르기 = 의도적 제스처
- ✓ 편집 모달에 명시적 삭제 버튼
- ✓ 카드가 깔끔해지고 집중도 ↑

삭제는 쉬워야 한다. 하지만 "실수로 쉬운" 것과 "의도적으로 쉬운" 것은 다르다.

Core Feature 01

기록 조각

빠른 기록 — 지금 이 순간을 3초 안에

- 결(카테고리) 선택 → 텍스트 입력 → 저장 3단계 완성
- 오늘의 속삭임:
 - 랜덤 프롬프트로 기록 시작을 돕는다
- 사진 · 파일 첨부 기능으로 기록 확장
- 녹음은 향후 확장예정
- 작성한 기록을 홈에서 바로 확인
- 오늘 날짜 기록만 홈에 표시 — 집중도 유지

💬 "완성된 글보다 시작할 수 있는 한 줄이 더 중요하다"

MyQuickNote - PRM Portfolio

나의 생각 조각

오늘 스친 아이디어를 놓치지 마세요

아이디어 스위치

지금 막 스친 그 생각, 적어볼까요?

중요

GitHub 업로드 후 주소 확인하기

오전 10:51

오늘 기분

기분: 신남 75점 / 아이디어가 잘 나옴

오전 10:51

나에게 한마디

완벽하게 안해도 된다

오전 10:51

할일

장보기

오전 10:51



사진



녹음



파일

반짝 아이디어

메모

영감

할일

나에게 한마디

오늘 기분

중요



아이디어 적어보기...

저장



기록 조각



아이디어 달력



생각 주머니

18 / 32

아이디어 달력

달력 회고 — 기록 있는 날짜에 점이 찍힌다

- 기록이 있는 날짜에 파란 점(·) 자동 표시
- 날짜 선택 시 그날의 조각만 필터링해서 보여준다
- 빈 날짜도 실패처럼 보이지 않게 설계
- "이날은 아직 기록이 없어요" — 부정적 표현 배제

"기록하지 않은 날도 실패가 아니라, 그냥 빈 날이다"

아이디어 달력

6월 2026

일	월	화	수	목	금	토
31	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	1	2	3	4

선택한 날의 기록 9

- GitHub 업로드 후 주소 확인하기

오전 10:51 · 중요
- 기분: 신남 75점 / 아이디어가 잘 나옴

오전 10:51 · 오늘 기분
- 완벽하게 안해도 된다

오전 10:51 · 침착하기
- 장보기 살 목록 정리하기 1. 세제 2. 양파 3. 우유

오전 10:51 · 할일

완료
- 작은 습관은 조용히 삶의 모양을 바꾼다

오전 10:50 · 영감

오늘 보 레퍼러스 · 히새 배겨 · 나라 표이트 카드뉴스

기록 조각

아이디어 달력

생각 주머니

19 / 32

마음 서랍

결별 아카이브 — 7개의 결로 모아보기

- 아이디어 · 메모 · 영감 · 할일 · 한마디 · 기분 · 중요
- 7개 고정 — Miller's Law 기반 인지 부담 최소화
- 결 클릭 시 해당 카테고리만 필터링
- 색상 코드로 직관적 구분

"7개로 고정 — 선택지가 많아지면 분류 자체가 부담이 된다"

생각 주머니

성격별로 모인 기록을 확인해 보세요



2개

반짝 아이디어
떠오른 생각과 기획 씨앗

모아보기



2개

메모
바로 적어둔 짧은 기록

모아보기



1개

영감
좋았던 문장과 장면

모아보기



1개

할일
준비중, 진행중, 완료 기록

모아보기



1개

나에게 한마디
스스로에게 남긴 응원

모아보기



1개

오늘 기분
하루의 감정과 상태

모아보기



1개

중요

모아보기

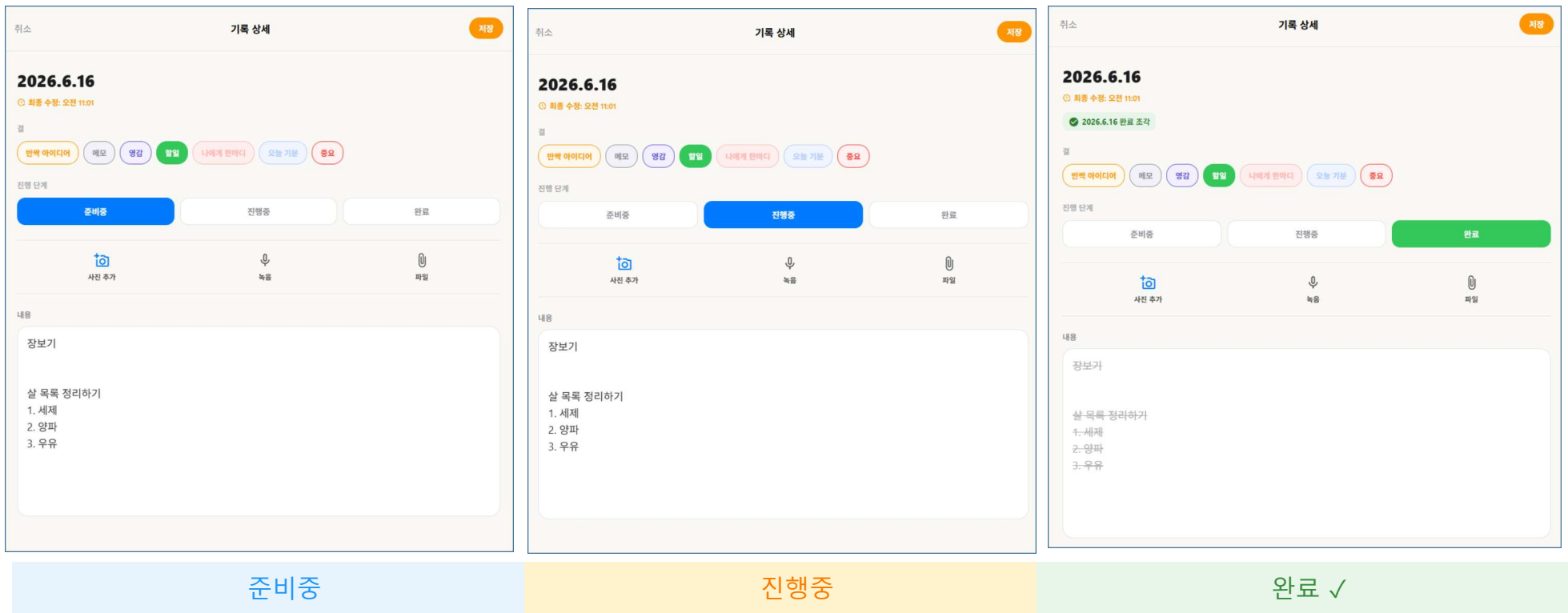
 기록 조각

 아이디어 달력

 생각 주머니

할일 상태 관리

할일일 때만 준비중 → 진행중 → 완료 3단계 상태 UI 노출



💡 기능은 많이 넣을 수 있다 — 하지만 "언제 보여줄지"가 프로덕트 센스다

어떻게 성공을 측정할 것인가

북극성 지표 · KPI · Anti-Metric 3층 구조

☆ NORTH STAR METRIC

일주일 내 다시 돌아와 조각을 남기는 사용자 비율 (W1 Retention)

"메모 앱의 본질은 계속 쓰는 것. 재방문율이 모든 걸 말한다."

핵심 KPI

40%

W1 Retention

가입 후 7일 재방문

1.5+

세션당 조각

한 번 쓸 때 담는 수

5%+

최소 카테고리

가장 적게 쓰는 걸 사용률

25%

아카이브 진입

"마음 서랍" 탭 방문

⚠ ANTI-METRIC · 올라가면 오히려 경고 신호

- 세션당 체류시간 10분 초과 → 앱이 무거워진 신호 (목표는 3초 앱)
- 일부 카테고리 70% 초과 → 걸 구조 불균형
- 알림 후 재방문 상승 → 원칙 5번(재촉하지 않기) 위반

앞으로의 길

MVP → UX 고도화 → 데이터 → 동기화 → 감성 확장 → IP 커머스

P1	P2	P3	P4	P5	P6
MVP	UX 고도화	데이터	동기화	감성 확장	IP · 커머스
7개 결 · CRUD · 3탭 구조	감성 네이밍 · 바텀시트 · 할일 상태	검색 · 태그 · 내보내기	클라우드 백업 · 로그인	테마 · 위젯 · 주간 리포트	로고몽 · 굿즈 · 펀딩
✓ 완료	✓ 완료	→ 진행중	계획	계획	계획

💰 B M 방 향

디지털 Freemium (월 2,900 원) — 이미지 무제한 · 클라우드 백업 · 전용 테마

IP · 굿즈

마스코트 "로고몽" — 인형 · 스테이셔너리 · 가족 소통 키트 (앱 재진입 트리거)

배운 6가지 — 제품과 기술

1인 풀스택 프로젝트에서 체득한 제품 감각 + 개발자와 소통하는 언어

🔗 PRODUCT

01 용어 하나가 프로젝트 톤을 바꾼다

"카테고리→결", "저장→담기"만 바꿨는데 사용자 감정이 달라짐. 기능 설계 전 용어 설계.

02 기능을 언제 보여줄지가 PM의 실력

할일 상태를 조건부로 숨긴 결정이 프로젝트 밀도를 결정. 더하기보다 숨기기가 어렵다.

03 MVP는 빠를수록 더 많이 배운다

6주 기획보다 2주 구현이 10배 더 배움. 본인이 사용자일 때 보이는 문제가 가장 정확.

🔗 ENGINEERING

01 파일 기반 라우팅의 원리

Expo Router에서 루트는 Stack, 탭은 Tabs. 이 구분이 안 되면 탭바가 두 줄로 쌓인다.

02 레이스 컨디션 방어

AsyncStorage 로드 전 useEffect가 먼저 발동하면 데이터가 덮어써짐. isLoading 플래그로 차단.

03 날짜 타임존 함정

new Date('2026-04-23') 은 UTC 해석. '/' 구분자로 바꿔야 로컬 시간대로 파싱된다.

PM은 코드를 짜지 않아도, "개발자의 언어"를 이해하면 훨씬 좋은 파트너가 된다.

다시 한다면 바꿀 3가지

자기 비판 = 성장 잠재력의 증거

01

사용자 인터뷰를 먼저 했을 것

✕ BEFORE

혼자 기획 → 구현 → 검증 순서

✓ NEXT

다음엔 최소 5명 인터뷰부터 시작

02

분석 도구를 처음부터 심었을 것

✕ BEFORE

지금은 추측 기반의 KPI

✓ NEXT

다음엔 Amplitude / PostHog 연동부터

03

경쟁 앱 Teardown 보고서를 만들었을 것

✕ BEFORE

왜 기존 앱들이 실패했는지 알게 파악

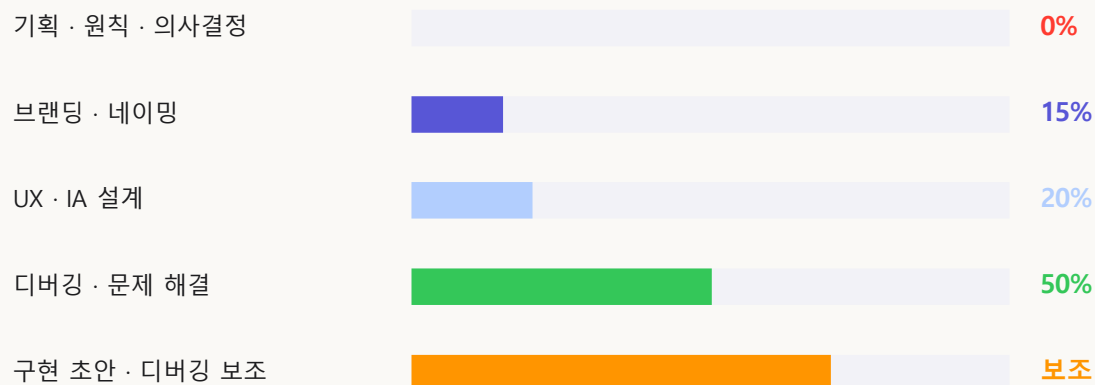
✓ NEXT

다음엔 상세 Teardown → 차별화 정교화

AI 협업 — 무엇을 맡기고, 무엇을 지켰나

2026년의 PM에게 필요한 것은 "AI 활용도"가 아니라 "AI 경계 설계 능력"

AI 기여도 (by 개발 단계)



☆ 의사결정 = 0% — 제품 철학과 원칙은 100% 본인 판단

Human-in-the-Loop · 4가지 핵심 판단

01 "결" 네이밍

AI: "카테고리"/"태그" 제안 → 본인: "결"로 재정의

02 오늘날 필터링

AI: 3가지 옵션 비교 → 본인: "집중의 설계" 관점 선택

03 할일 조건부 UI

AI: 전체 상태 UI 구조 → 본인: 조건부로만 노출 결정

04 메인+서브 설계

AI: 3가지 옵션 → 본인: 제품 원칙 기준 A안 채택

AI는 구현 초안 · 디버깅 보조로 활용 — 제품 원칙 · 우선순위 · 최종 판단 · 검증은 직접 수행

코드로 증명하는 제품 의사결정 4가지

코드 한 줄이 기획 의도를 실현하는 순간 — 작은 결정이 제품의 사용감을 만든다

01 기록은 사라지면 안 된다

→ `AsyncStorage` + `isLoading` 패턴

앱 종료 후에도 기록이 남아야 신뢰가 쌓인다

02 기록은 하나의 조각으로 관리된다

→ `content` · `category` · `date` · `files` · `status` 구조화

확장 가능한 설계가 유연한 제품을 만든다

03 마음 서랍은 선택한 결만 보여준다

→ `category` 기반 필터링

골라보는 경험이 아카이브를 의미있게 만든다

04 할일은 상태가 필요할 때만 보여준다

→ 조건부 UI + `completedDate`

기능은 필요한 순간에만 존재해야 한다

💡 기획을 코드로 증명하는 PM — 개발자와 같은 언어로 말할 수 있다

Code 01.

기록 저장과 불러오기

AsyncStorage로 로컬 저장 — 앱 종료 후에도 데이터 유지

AsyncStorage 패턴

getItem → 기록 불러오기
 setItem → 기록 저장
 isLoading → 로드 전 저장 방지

code

```

45 const [selectedNote, setSelectedNote] = useState<any>(null);
46 const [todayQuestion, setTodayQuestion] = useState('');
47 const [isLoading, setIsLoaded] = useState(false);
48
49 useEffect(() => {
50   loadNotes();
51   setTodayQuestion(questions[Math.floor(Math.random() * questions.length)]);
52 }, []);
53
54 useEffect(() => {
55   if (isLoading) {
56     saveNotes(notes);
57   }
58 }, [notes, isLoading]);
59
60 const loadNotes = async () => {
61   try {
62     const data = await AsyncStorage.getItem('@my_notes');
63     if (data) {
64       setNotes(JSON.parse(data));
65     }
66   } catch (error) {
67     console.error(error);
68   } finally {
69     setIsLoaded(true);
70   }
71 };
72

```

💡 앱을 종료해도 기억하는 앱 — 신뢰는 데이터에서 시작된다

Code 02.

기록 데이터 구조 설계

텍스트 · 카테고리 · 날짜 · 이미지 · 파일 · 할일 상태를 하나의 객체로

 Note 인터페이스

content / category → 핵심 내용과 걸
 date / time → 달력 뷰와 시간 표시
 imgs / files → 아카이브 확장
 status / completedDate → 할일 조건부 적용

code

```

106 const addNote = () => {
107   if (
108     text.trim().length === 0 &&
109     tempImages.length === 0 &&
110     tempFiles.length === 0
111   ) {
112     return;
113   }
114
115   const now = new Date();
116   const newNote = {
117     id: Date.now(),
118     content: text,
119     category: currentCategory,
120     date: `${now.getFullYear()}.${now.getMonth() + 1}.${now.getDate()}`,
121     time: now.toLocaleTimeString('ko-KR', {
122       hour: '2-digit',
123       minute: '2-digit',
124       hour12: true,
125     }),
126     imgs: tempImages,
127     files: tempFiles,
128     status: '준비중',
129     completedDate: null,
130     lastEdited: null,
131   };
132
133   setNotes([newNote, ...notes]);
134   setText('');
135   setTempImages([]);
136   setTempFiles([]);
137   setIsAttachOpen(false);
138   Keyboard.dismiss();
139 };
140

```

💡 하나의 조각이 담을 수 있는 것들 — 설계가 확장성을 만든다

Code 03.

카테고리별 기록 필터링

선택한 결 기준으로 전체 기록 중 해당 카테고리만 모아서 표시

🔍 동적 라우팅 필터링

useLocalSearchParams → URL에서 결 이름 수신
 currentCategory → 안전한 타입 변환
 .filter() → 해당 카테고리 기록만 추출

code

```

73  const saveNotes = async (data: any[]) => {
74    try {
75      await AsyncStorage.setItem('@my_notes', JSON.stringify(data));
76    } catch (error) {
77      console.error(error);
78    }
79  };
80
81  const pickImage = async () => {
82    const result = await ImagePicker.launchImageLibraryAsync({
83      mediaTypes: ImagePicker.MediaTypeOptions.Images,
84      allowsMultipleSelection: true,
85      quality: 0.7,
86    });
87
88    if (!result.canceled) {
89      setTempImages([...tempImages, ...result.assets.map((asset) => asset.uri)]);
90      setIsAttachOpen(false);
91    }
92  };
93
94  const pickFile = async () => {
95    const result = await DocumentPicker.getDocumentAsync({
96      copyToCacheDirectory: true,
97      multiple: true,
98    });
99
100    if (!result.canceled) {
101      setTempFiles([...tempFiles, ...result.assets]);
102      setIsAttachOpen(false);
103    }
104  };
105
106  const addNote = () => {
107    if (
108      text.trim().length === 0 &&
109      tempImages.length === 0 &&
110      tempFiles.length === 0
111    ) {

```

💡 마음 서랍은 골라보는 공간 — 필터링이 탐색 UX를 만든다

Code 04.

할일 상태 관리

준비중·진행중·완료 상태 관리 — 완료 시 날짜 자동 저장

✓ 상태 변경 + 완료일 기록

handleStatusChange

→ 상태 + 완료일 한번에 업데이트

completedDate

→ 완료 전환 시에만 날짜 기록

준비중 / 진행중 / 완료

→ 할일 카테고리일 때만 상태 UI 노출

code

```

106 const addNote = () => {
107   if (
108     text.trim().length === 0 &&
109     tempImages.length === 0 &&
110     tempFiles.length === 0
111   ) {
112     return;
113   }
114
115   const now = new Date();
116   const newNote = {
117     id: Date.now(),
118     content: text,
119     category: currentCategory,
120     date: `${now.getFullYear()}.${now.getMonth() + 1}.${now.getDate()}`,
121     time: now.toLocaleTimeString('ko-KR', {
122       hour: '2-digit',
123       minute: '2-digit',
124       hour12: true,
125     }),
126     imgs: tempImages,
127     files: tempFiles,
128     status: '준비중',
129     completedDate: null,
130     lastEdited: null,
131   };
132

```

💡 기능은 필요할 때만 보인다 — 조건부 UI가 앱의 밀도를 만든다

작은 조각들이 쌓이면
나를 이해하는 **자산**이 됩니다.

감사합니다

GitHub github.com/ebj0925-cyber/memo_2

Email kae8231@gmail.com

연락처 **010-2784-2509**